9). Composite Design Pattern.

Compose objects into tree structure to represent part-whole hierarchies. composite lets clients treat individual objects and compositions uniformly.

In simple terms:-
- Treat single object and group of objects the same way.
- work with tree-like structure easily.

Without Composite: it Leads to if-else, instanceof, complex client logic.

Core Idea

- Both leaf objects and composite objects
- Implements the same interface.
- client call methods without caring about structure.

Eg:- File System.
- File → single object.
- Folder → contains files and folders.
- Both supports operations like → • size()
  • delete().

- structure of composite pattern.
1. Component (common interface).
2. Leaf (individual object).
3. Composite (group of object).
4. Client.

Code

- Component Interface.

```
interface FileSystemItem {
    void showDetails();
}
```

- Leaf Class

```
class File implements FileSystemItem {
    private String name;
    File(String name) {
        this.name = name;
    }
    public void showDetails() {
        S.o.println("File" + name);
    }
}
```

- Composite class

```
class Folder implements FileSystemItem.
    private String name;
    private List<FileSystemItem> items = new ArrayList<>();

    Folder(String name) {
        this.name = name;
    }
    void add(FilesystemItem item) {
        items.add(item);
    }
    public void showDetails() {
        S.o.println("Folder." + name);
        for(FileSystemItem item: items) {
            item.showDetails();
        }
    }
}
```

Client code:

```
File file1 = new File("resume.pdf");
File file2 = new File("photo.png");

Folder folder = new Folder("Documents");

folder.add(file1);
folder.add(file2);

folder.showDetails();
```